Software Requirements Specification

For

Unbound: Fest Management Platform

Institution: MIT Academy of Engineering (MITAOE)

Project Goal: To architect a centralized, high-concurrency digital ecosystem for college fest management. The system leverages a robust relational database schema to manage Colleges, Clubs, Fests, and Student participation with absolute data integrity and zero redundancy.

Contents

1 Introduction **2**
1.1 Engineering Vision and Objective 2
1.2 User Hierarchy & Access Model 2

2 System Architecture **3**
2.1 The Full-Stack Technical Pillars 3

3 Technical Stack Matrix **4**

4 Functional Requirements **5**
4.1 Security & Identity (RBAC) 5
4.2 High-Precision Registration Logic 5
4.3 Attendance and Certification Module 5

5 Use Case Scenarios: User Interaction **6**
5.0.1 Use Case 1: Event Discovery and Registration (Solo) 6
5.0.2 Use Case 2: Team Formation and Member Joining 6

6 Use Case Scenarios: Administration **7**
6.0.1 Use Case 3: College-Level Content Management 7
6.0.2 Use Case 4: Event Orchestration and Club Management 7

7 External Integrations **8**
7.1 Relational Persistence Management (AWS RDS) 8
7.2 Financial Transaction Verification (Razorpay) 8

8 Non-Functional Requirements **9**
8.1 Scalability, Performance, and Concurrency 9
8.2 Security, Reliability, and Data Safety 9

9 Conclusion **10**
9.1 System Evolution and Roadmap 10
9.2 Final Summary 10

1 Introduction

The Unbound project is a strategic technical response to the fragmented administrative landscape currently governing technical and cultural fests at MITAOE. Current systems fail to provide a unified view of student participation across multiple colleges and clubs, leading to data silos and manual verification errors.

1.1 Engineering Vision and Objective

The primary objective of Unbound is to replace legacy manual workflows with a professional-grade, unified digital backbone. The vision focuses on relational consistency, ensuring that every entity—from a college record to an attendance log—is interconnected via primary and foreign keys. This prevents data duplication and ensures that certificates are issued only to verified participants.

1.2 User Hierarchy & Access Model

Unbound utilizes a multi-tenant user model stored in the users entity, where each user is mapped to a specific colleges record via a college_id. This hierarchy ensures that administrative actions are scoped correctly.

User Role	Primary Access Right	Core Responsibility
Super Admin	Global System Privileges	Global oversight across all colleges and system-wide audits.
College Admin	College-Level Privileges	Management of college-specific clubs, fests, and announcements.
Club Admin	Moderator Privileges	Organization of events within a specific clubs entity.
Student	Standard User Privileges	Event discovery, registrations, and teams participation.

Table 1: User Role Responsibility Matrix in Relational Context

2 System Architecture

Unbound utilize a Three-Tier Decoupled Architecture, separating the Presentation, Application, and Data layers to ensure fault isolation and high-throughput processing.

2.1 The Full-Stack Technical Pillars

The architecture is structured around four specialized pillars to maintain the platform's motto of resilience and precision:

- **Presentation Layer (Next.js 16):** Acts as the edge-rendered interface for event discovery. It utilizes Server-Side Rendering (SSR) to ensure that highly dynamic data—such as real-time *max_participants* availability—is accurate at the moment of request, eliminating "stale data" issues common in client-side portals.
- **Application Layer (Spring Boot 3.5.11):** Serves as the central orchestrator for business logic and relational integrity. It manages complex multi-threaded transactions and enforces the primary gatekeeping rules. ***Example of Validation Logic:*** The backend performs a relational check to ensure a student's `user.college_id` matches the target `fest.college_id` before authorizing a transaction in the *Registrations* entity. Registration is blocked if institutional scopes do not align.
- **Data Persistence (PostgreSQL 15):** A robust relational engine maintaining the integrity of all 12 system entities. It utilizes specialized constraints to manage One-to-Many ($1 : N$) and Many-to-Many ($M : N$) mappings, ensuring that relational keys like *registration_id* are never orphaned during cascading updates.
- **Cloud Infrastructure (AWS Fargate):** A serverless container orchestration environment that provides the elastic backbone for the Spring Boot services. It automatically scales horizontal pods during peak registration windows to maintain sub-300ms latency.

3 Technical Stack Matrix

The stack was chosen to ensure that the relational complexity of the Unbound schema is handled with high-performance I/O and strict type-safety.

Component	Technology	Version	Engineering Justification
Language	Java	17	Ensures type-safety between the database schema and API.
Backend Framework	Spring Boot	3.5.11	Robust JPA/Hibernate integration for managing the 12 entities.
Frontend Framework	Next.js	16.x	Server Components for optimized data fetching from RDS.
Database Engine	PostgreSQL	15.17	Native support for relational constraints and complex joins.
Cloud Orchestration	AWS Fargate	Latest	Serverless execution environment for containerized services.
Payment Gateway	Razorpay SDK	1.4.3	Integrated with the registrations status update flow.

Table 2: Consolidated Technical Stack Matrix

4 Functional Requirements

The functional requirements are mapped directly to the system workflow, ensuring a seamless transition from college-level management to student-level certification.

4.1 Security & Identity (RBAC)

Authentication is managed via the users entity. Every session must validate the role field (Student, Club Admin, College Admin, Super Admin). This RBAC model prevents unauthorized access to the announcements publishing module and the certificates issuance gateway.

4.2 High-Precision Registration Logic

The registrations entity acts as the bridge for the Many-to-Many relationship between users and events.

- **Transaction Integrity:** When a student registers, the system checks the max_participants in the events table using pessimistic locking to prevent race conditions.
- **Relational Integrity:** Validation logic ensures that students cannot bypass the college_id restriction through direct API calls.
- **Group Participation:** For team-based events, a record is first created in the teams table, followed by individual mappings in the team members table to track user-team relationships.

4.3 Attendance and Certification Module

Post-event, the system utilizes the attendance entity to verify physical presence. Only users with a present status in the attendance table and a completed status in the registrations table are eligible for a record in the certificates entity, which stores the certificate_url.

5 Use Case Scenarios: User Interaction

Student interactions focus on the discovery and participation modules of the relational system.

5.0.1 Use Case 1: Event Discovery and Registration (Solo)

Actor: Student (User)
Precondition: User is authenticated; user's college_id matches event's college_id; capacity exists.
Flow: Student browses events filtered by categories and fests → Clicks "Register" → System creates a record in registrations → Status set to Pending → Payment success → Status updated to Confirmed.
Postcondition: Student is mapped to the event; registration log is secured.
Exception: 403 Forbidden if college_id mismatch; 409 Conflict if max_participants limit is reached.

5.0.2 Use Case 2: Team Formation and Member Joining

Actor: Student (Team Leader)
Precondition: User is authenticated; event allows team-based participation.
Flow: Student creates a new record in the teams table → Team name assigned → Other users join via the team members mapping table → System validates all members belong to the same college_id.
Postcondition: Group participation is established with a unique team_id.
Exception: 404 Not Found if team_id is invalid or expired.

6 Use Case Scenarios: Administration

Administrative workflows involve managing the organizational entities: colleges, clubs, and fests.

6.0.1 Use Case 3: College-Level Content Management

Actor: College Admin
Precondition: Authenticated with college_admin role.
Flow: Admin publishes a message to the announcements table → System maps the announcement to the specific college_id → Admin creates a new fests record for the upcoming season.
Postcondition: Students belonging to the same college see new fests and announcements.
Exception: 403 Forbidden if admin attempts to publish to a different college_id.

6.0.2 Use Case 4: Event Orchestration and Club Management

Actor: Club Admin
Precondition: Authenticated with club_admin role.
Flow: Admin accesses the clubs dashboard → Creates a new record in the events table → Assigns a category_id, fest_id, and club_id → Venue and Date scheduled.
Postcondition: The event is live and linked to the broader festival relational structure.
Exception: 400 Bad Request if mandatory relational keys are missing.

7 External Integrations

Unbound integrates with cloud-native services to handle data persistence and security.

7.1 Relational Persistence Management (AWS RDS)

The system utilizes AWS RDS for PostgreSQL to host the 12 core entities.

- **Schema Integrity:** Foreign key constraints are enforced at the database level to ensure that deleting a colleges record cascades or restricts updates to associated clubs and users.
- **Scalability:** Read-replicas are utilized to handle high-frequency GET requests to the events and announcements tables during active fest weeks.

7.2 Financial Transaction Verification (Razorpay)

Razorpay webhooks are mapped to the registrations table. The system ensures that a student is only granted a registration_id with a Paid status after an asynchronous HMAC-signature check verifies the transaction.

8 Non-Functional Requirements

The platform is designed to maintain 99.9% uptime while protecting the sensitive relational data of thousands of students.

8.1 Scalability, Performance, and Concurrency

- **Concurrency Handling:** The system must process 500 to 600 concurrent inserts into the registrations and attendance tables without database row-locking delays.
- **Query Latency:** Complex JOIN queries involving users, teams, and team members must execute in sub-300ms.

8.2 Security, Reliability, and Data Safety

- **Data Security:** All passwords in the users entity are hashed with BCrypt. Data in transit is encrypted via TLS 1.3.
- **Availability:** Multi-AZ deployment for PostgreSQL ensures that the relational hierarchies are never compromised by hardware failure.
- **Backup Policy:** Automated daily snapshots of the full database schema with a 30-day retention period for disaster recovery.

9 Conclusion

The Unbound platform represents a significant digital transformation for the MIT Academy of Engineering. By implementing a highly structured relational database comprising 12 specialized entities—from colleges to certificates—the platform solves the long-standing issues of data redundancy and fragmented administrative oversight. The architecture ensures that every student registration, team formation, and attendance log is accurately tracked and verifiable.

9.1 System Evolution and Roadmap

As the platform scales, the relational design allows for seamless additions such as AI-driven event recommendations and automated volunteer management without disrupting the core schema. The integration of AWS Fargate ensures that the system can adapt to the growing demands of multiple colleges within the MITAOE ecosystem.

9.2 Final Summary

In conclusion, Unbound provides a resilient, precise, and user-centric solution for fest management. By enforcing a strict Role-Based Access Control model and ensuring ACID compliance for every transaction, the system guarantees a stable environment for thousands of users. This document stands as the definitive technical baseline for the production release of Unbound.